



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

# Python IV: Functions and Algorithms
## Code Structure

IBI 1, Lecture 8.1

Dr Ying CHI – yingchi@intl.zju.edu.cn

Adapted from Dr Rob Young – robert.young@ed.ac.uk

Semester 2, 2025/26

# In Python III, we learnt:

- What a regular expression is
- re module – `re.search()` / `re.match()` / `re.findall()` …
- File operations – `open()` / `write()` / `close()` …
- pandas module

# Today we are going to learn

- Functions
- How to create and access abstract data types through classes
- `Coordinate` example

# Learning objectives

## After this lecture, you should be able to:

- Write simple functions in Python
- Define and explain classes

# Functions

- **Functions** are reusable pieces/chunks of code.
- Functions are not run in a program until they are "**called**" or "**invoked**" in a program.
- Function characteristics:
  - Has a **name**
  - Has **parameters** (0 or more)
  - Has a **docstring** (optional but recommended)
  - Has a **body**
  - **Returns** something

# What does a function look like?

```
def is_even (i)
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print ("inside is even")
    return i%2 == 0

is_even(3)
```

# What does a function look like?

keyword

name

parameters or arguments

```
def is_even (i):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print ("inside is even")
    return i%2 == 0

is_even(3)
```

# What does a function look like?

keyword

name

parameters or arguments

```
def is_even (i)
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print ("inside is even")
    return i%2 == 0

is_even(3)
```

specification, docstring

body

# What does a function look like?

keyword

name

parameters or arguments

```
def is_even (i)
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print ("inside is even")
    return i%2 == 0
```

specification, docstring

body

```
is_even(3)
```

Later in your code, you call the function using its name and values for parameters

# Inside the function body

```
def is_even (i)
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    print ("inside is even")
    return i%2 == 0
```

run some commands

keyword

expression to evaluate and return

# Variable scope

- The formal parameter is assigned the value of the actual parameter…when the function is called.
- A new scope/frame/environment is created when you enter a function.
- The scope is the mapping of names to objects.

```
def f(x):
    x = x + 1
    print(' in f(x): x =', x)
    return x

x = 3
z = f(x)
```

formal parameter

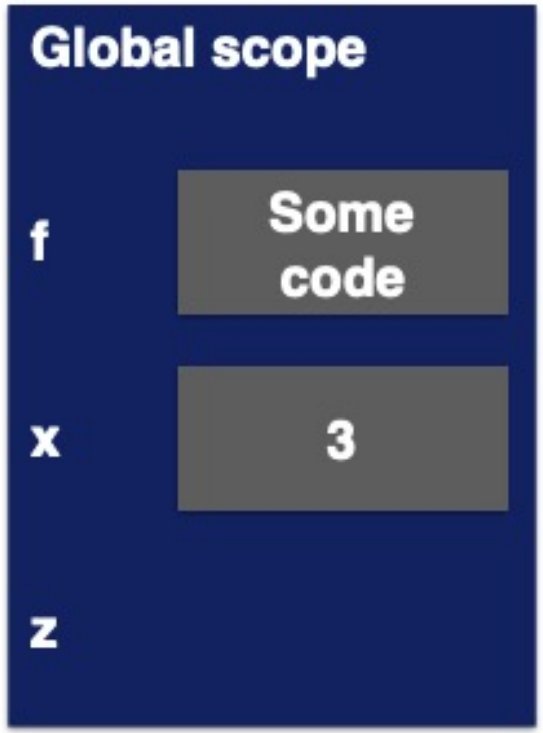
actual paramater

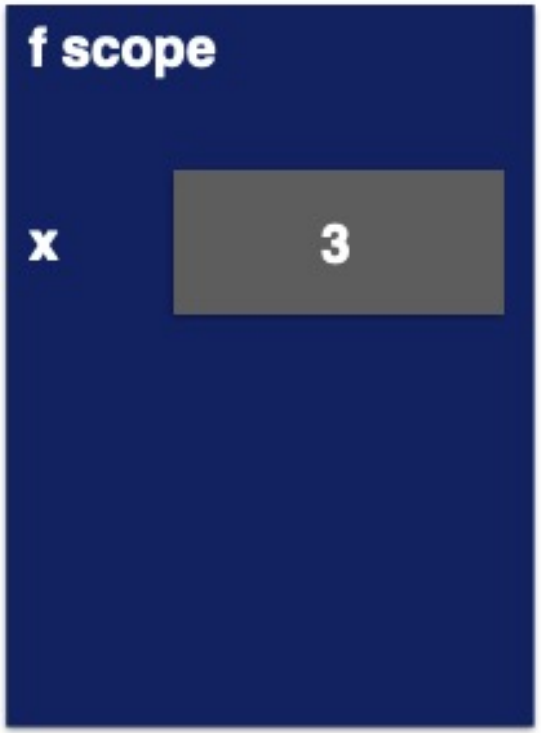
function definition

Main program code
- Initialises a variable x
- Makes a function call f(x)
- Assigns return of function to variable z

# Variable scope

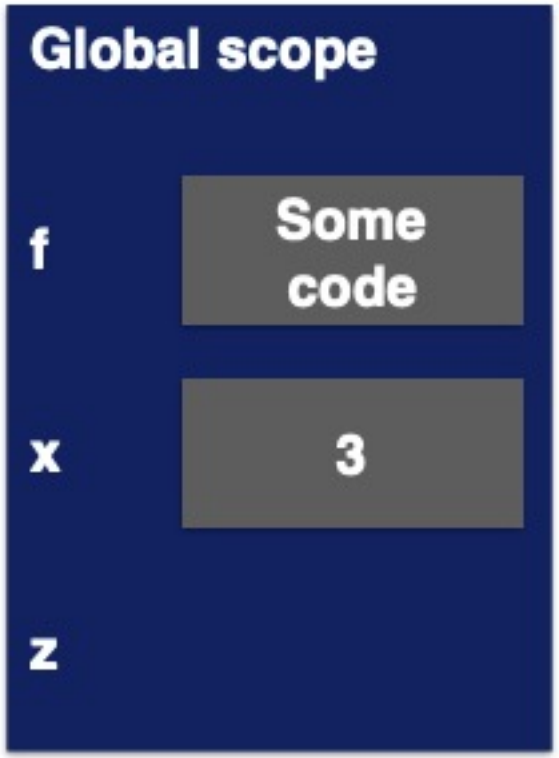
```
def f(x):
      x = x + 1
      print(' in f(x): x =', x)
      return x

x = 3
z = f(x)
```

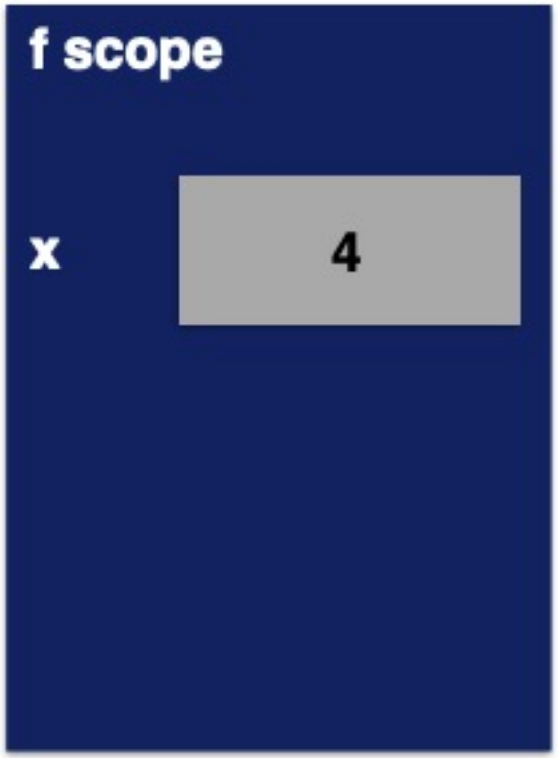


# Variable scope

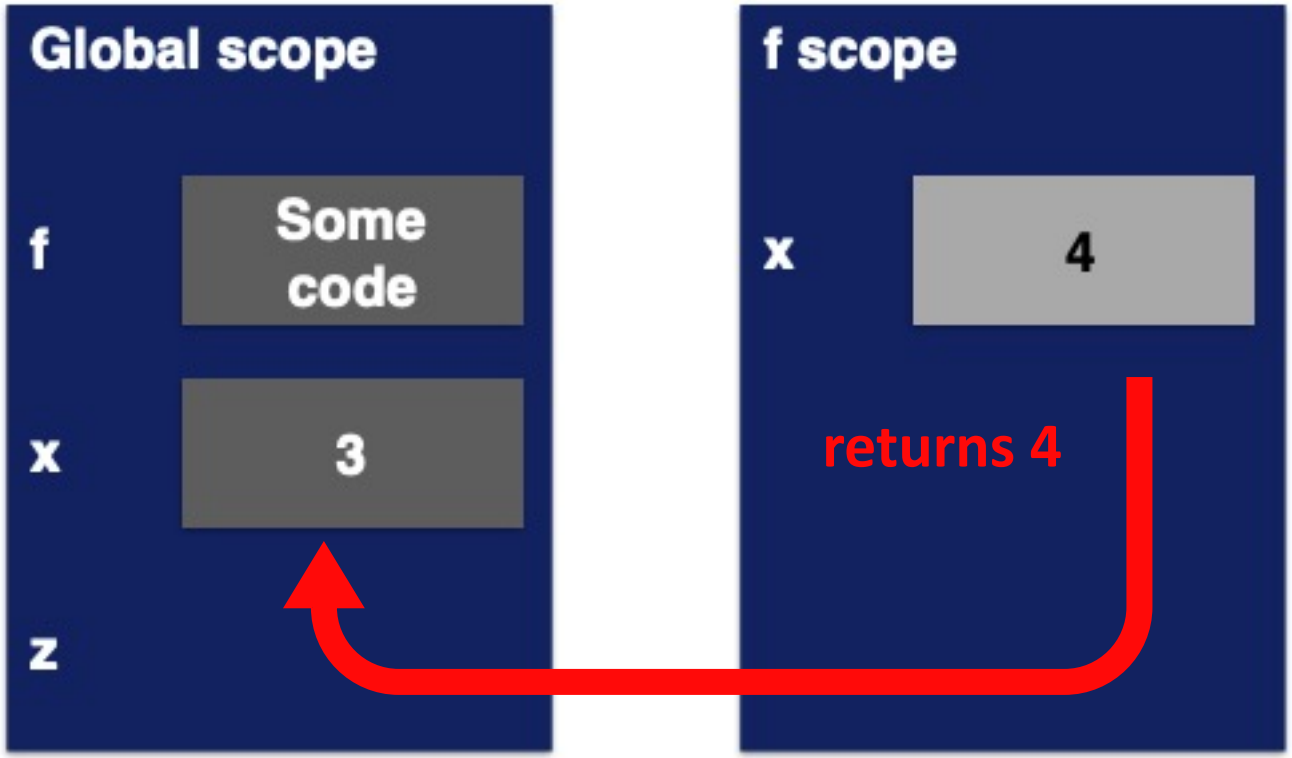
```
def f(x):
      x = x + 1
      print(' in f(x): x =', x)
      return x

x = 3
z = f(x)
```



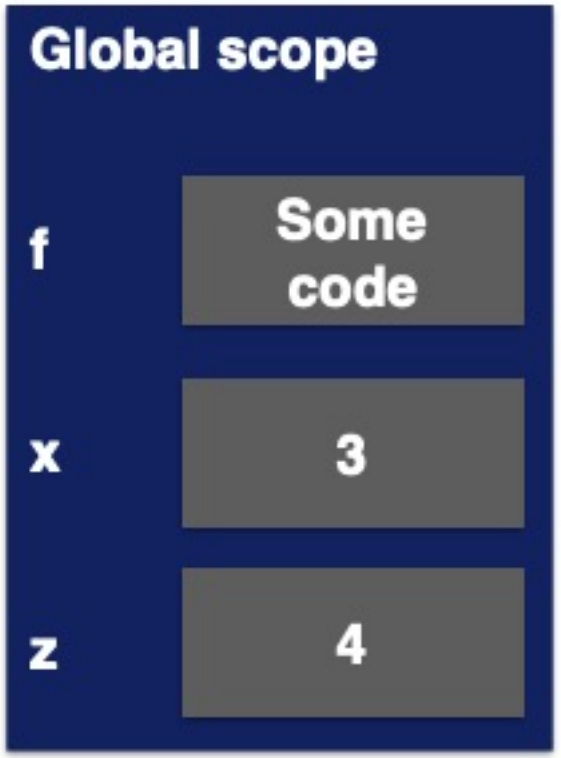
# Variable scope

```
def f(x):
      x = x + 1
      print(' in f(x): x =', x)
      return x

x = 3
z = f(x)
```



# Variable scope

```
def f(x):
      x = x + 1
      print(' in f(x): x =', x)
      return x

x = 3
z = f(x)
```



# A quick warning if you forget to return

```
def is_even(i):
    """
    Input: i, a positive int
    Does not return anything
    """
    i%2 == 0
```

without a return statement

- Python returns the value None, if no return given.
- Represents the absence of a value.

# `return` vs. `print`

- Return only has meaning **inside** a function.
- Only **one** return can be executed inside a function.
- Code inside function but after return statement is not executed.
- Has a value associated with it, **given to the function caller**.

- Print can be used **outside** functions.
- Can execute **many** print statements inside a function.
- Code inside function can be executed after a print statement.
- Has a value associated with it, **outputted** to the console.

# Another example of variable scope

- Inside a function, you **can access** a variable defined outside.
- Inside a function, you **cannot modify** a variable defined outside.
- Can use **global variables**, but this is frowned up → get in a mess.

```
def f(y):
    x = 1
    x += 1
    print(x)

x = 5
f(x)
print(x)
```

x is redefined in scope of f

different x subject

# Another example of variable scope

- Inside a function, you **can access** a variable defined outside.
- Inside a function, you **cannot modify** a variable defined outside.
- Can use **global variables**, but this is frowned up → get in a mess.

```
def f(y):
    x = 1
    x += 1
    print(x)

x = 5
f(x)
print(x)
```

x is redefined in scope of f

different x subject

```
def g(y):
    print(x)
    print(x + 1)

x = 5
g(x)
print(x)
```

x from outside g

x inside g is picked up from scope that called function g

# Another example of variable scope

- Inside a function, you **can access** a variable defined outside.
- Inside a function, you **cannot modify** a variable defined outside.
- Can use **global variables**, but this is frowned up → get in a mess.

```
def f(y):
    x = 1
    x += 1
    print(x)

x = 5
f(x)
print(x)
```

x is redefined in scope of f

different x subject

```
def g(y):
    print(x)
    print(x + 1)

x = 5
g(x)
print(x)
```

x from outside g

x inside g is picked up from scope that called function g

```
def h(y):
    x += 1

x = 5
h(x)
print(x)
```

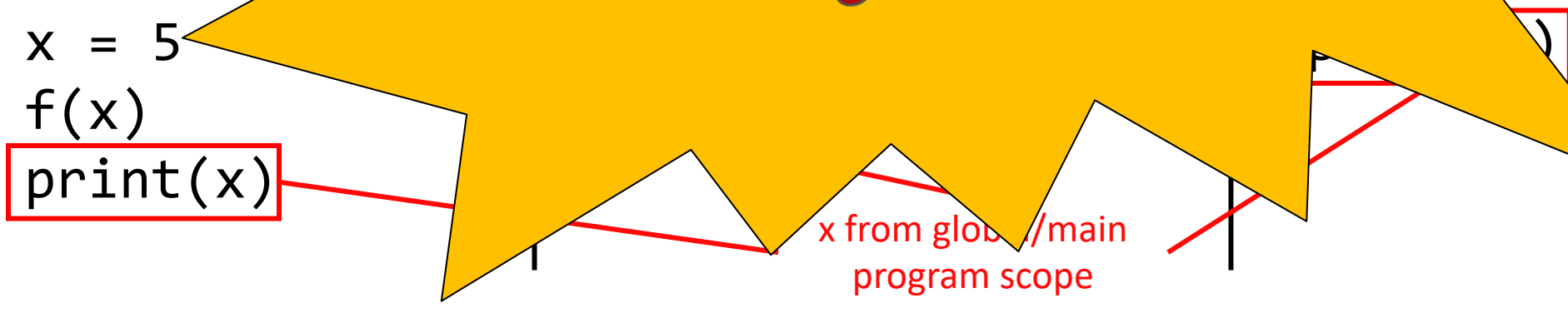
UnboundLocalError: local variable 'x' referenced before assignment

# Another example of variable scope

- Inside a function, you ess a v de.
- Inside a function, you outside
- Can use gl .

```
def f(y):
    x
    print

x = 5
f(x)
print(x)
```

Try it out for yourself and see what works each time you write code!!



# Creating and using your own data types with classes

- Need to make a distinction between **creating a class** and **using an instance** of the class.
- **Creating** the class involves:
  - Defining the class name
  - Defining class attributes
  - For example, someone wrote code to implement a list class
- **Using** the class involves:
  - Creating new **instances** of objects
  - Doing operations on the instances
  - For example, L=[1,2] and len(L)

# Define your own data types

- Use the class keyword to define a new type

class definition  name/type  class parent

```
class Coordinate (object):
    # define attributes here
```

- Similar to `def`, indent code to indicate which statements are part of the **class definition**
- The word object means that Coordinate is a Python object and **inherits** all its attributes
  - `Coordinate` is a subclass of `object`
  - `object` is a superclass of `Coordinate`

# Instance of a class

- First have to define how to create an instance of an object.
- Use a special method called __init__ to initialise some data attributes.

```
class Coordinate (object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

what data initialises a Coordinate object

special method to create an instance __ is double underscored

parameter to refer to an instance of the class

two data attributes for every Coordinate object

# Actually creating an instance of a class

```
c = Coordinate(3,4)
origin = Coordinate (0,0)
print(c.x)
print(origin.x)
```

create a new object of type `Coordinate` and pass in 3 and 4 to the `__init__`

use the dot to access an attribute of instance c

- Data attributes of an instance are called **instance variables**.
- Don't provide argument for `self`, Python does this automagically.

# What is a method?

- A procedural attribute, like a **function that works only with this class**.
- Python always passes the object as the first argument.
- Convention is to use **self** as the name of the first argument of all methods.
- The "**.**" **operator** is used to access any attribute
  - A data attribute of an object
  - A method of an object

# Let's define a method for the `Coordinate` class…

```
Class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def distance (self, other):
        # put your code in here
```

use it to refer to any instance

another parameter to method

- Other than `self` and dot notation, methods behave just like functions (take parameters, do operations, return)

# Let's define a method for the Coordinate class…

```
Class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def distance (self, other):
        x_diff_sq = (self.x-other.x)**2
        y_diff_sq = (self.y-other.y)**2
        return (x_diff_sq + y_diff_sq)**0.5
```

use it to refer to any instance

another parameter to method

dot annotation to access data

- Other than `self` and dot notation, methods behave just like functions (take parameters, do operations, return)

# How do you *actually* use a method?

```
def distance (self, other):
    # code here
```

method def

Using the class:

Conventional way:

```
c = Coordinate(3,4)
zero = Coordinate(0,0)
print(c.distance(zero))
```

object to call method on

name of method

parameter not including `self` (`self` is implied to be c)

Is equivalent to:

```
c = Coordinate(3,4)
zero = Coordinate(0,0)
print(Coordinate.distance(c, zero))
```

name of class

name of method

parameters, including an object to call the method of, representing `self`

# Summary

- A **function** is a piece of code that performs a task of some kind
  - Reducing duplication of code
  - Information hiding
  - Routine processing of tasks, e.g. writing files of a consistent format
- **Classes** make it easy to reuse code
  - Many Python modules define new classes
  - Each class has a separate environment (no collision on function names)
  - Inheritance allows subclasses to redefine or extend a selected subset of a superclass' behaviour.

# Super/father class example :

```
class Animal:  # This is a Super Class
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} makes a sound.")

class Dog(Animal):  # Dog inherited from Animal. Animal is Dog's Super Class
    def speak(self):  # The sub-class content of super class can be revised
        print(f"{self.name} says: Woof!")

my_dog = Dog("Buddy")
my_dog.speak()  # Output: Buddy says: Woof!
# But my_dog still has super class' name property
print(my_dog.name)  # Output: Buddy
```

# Learning objectives

## Now, you should be able to:

- Write simple functions in Python
- Define and explain classes



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

# Python IV: Functions and Algorithms
# Code Structure
# Any questions?

IBI 1, Lecture 8.1

Ying CHI, adapted from Dr Rob Young – robert.young@ed.ac.uk

Semester 2, 2025/26